For bkhack, the user gets to familiarize with the concept of *stream-based programming*. Indeed, visitors of the site will often catch glance of command hints and tips as they are littered about the user interface. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Stream programming in bkhack is sh pipelining, with certain flavorful differences. The most important difference is that, instead of dealing with textual data and lines and files, here we deal with abstract post data and post counts. Consider

```
feed | split -c 10
```

where `split` has the option `-c NUM` where `NUM` is the count size of each chunk. Conventionally, the POSIX `split` command accepts the option `-l NUM` where `NUM` is line number size of chunk.

Another difference is that there is no filesystem, no concept of space. There is a command for every page, but there's no way to navigate "relatively" such as going "back" or "forth"; in other words, there's no `cd` command.

Pipeline

Introduced by Doughlas, pipelining enables commands to compose with each other parsibly simply via textual data. In this composition, there are relationships between commands to form the pipeline by parts.

The *source* is the start of the pipeline. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere. The rest are *cantraps*–commands that.

It's worth noting that, even on the conceptual level, the evaluation order of the part commands is that all commands start simultaneously when a pipeline runs.

Function

A reusable pipeline or grouping of commands would be called a *function*. In the bkhack shell language,